

Formalização de diferentes noções de ordenação

Carolina Campos - 221030830

Gustavo Rinaldi - 222008664

11 de julho de 2026

1 Introdução

Equivalência entre diferentes noções de ordenação. O objetivo deste arquivo é formalizar e demonstrar a equivalência entre quatro definições distintas de listas ordenadas.

2 Desenvolvimento

2.1 Definições de ordenação e propriedades básicas

2.1.1 Ordenação 1 (ord1)

A primeira definição de ordenação, chamada **ord1**, é uma definição indutiva contendo 3 regras de formação:

$$\frac{}{ord1\ nil} (ord1_nil) \quad \frac{}{ord1\ (x :: nil)} (ord1_one) \quad \frac{x \leq y \quad ord1(y :: l)}{ord1\ (x :: y :: l)} (ord1_all)$$

```
Inductive ord1 : list nat → Prop :=  
| ord1_nil: ord1 nil  
| ord1_one: ∀ x, ord1 (x::nil)  
| ord1_all: ∀ l x y, x ≤ y → ord1 (y::l) → ord1 (x::y::l).
```

2.1.2 Ordenação 2 (ord2)

A segunda definição de ordenação, chamada **ord2**, é uma definição indutiva contendo 2 regras de formação:

$$\frac{}{ord2\ nil} (ord2_nil) \quad \frac{x \leq^* l \quad ord2\ l}{ord2\ (x :: l)} (ord2_all)$$

onde $x \leq^* l$ significa que x é menor ou igual que todo elemento da lista l . Formalmente, este predicado é definido a seguir:

Definition $le_all\ x\ l := \forall y, In\ y\ l \rightarrow x \leq y$.

```
Inductive ord2 : list nat → Prop :=  
| ord2_nil: ord2 nil  
| ord2_all: ∀ l x, x ≤* l → ord2 l → ord2 (x::l).
```

2.1.3 Ordenação 3 (ord3)

A terceira definição de ordenação, chamada **ord3**, diz que uma lista está ordenada se cada elemento é menor ou igual ao elemento seguinte:

Definition *ord3* (*l* : *list nat*) : **Prop** :=

$\forall i, \text{length } l > 1 \rightarrow (S\ i) < \text{length } l \rightarrow \text{nth } i\ l\ 0 \leq \text{nth } (S\ i)\ l\ 0.$

Lemma *ord3_nil*: *ord3 nil*.

Lemma *ord3_one*: $\forall x, \text{ord3 } (x::\text{nil}).$

Lemma *ord3_two*: *ord3* (*1::2::nil*).

2.1.4 Ordenação 4 (ord4)

A quarta definição de ordenação, chamada **ord4**, diz que uma lista está ordenada se cada elemento é menor ou igual que qualquer elemento que esteja em posição anterior:

Definition *ord4* (*l* : *list nat*) : **Prop** :=

$\forall i\ j, i < j \rightarrow j < \text{length } l \rightarrow \text{nth } i\ l\ 0 \leq \text{nth } j\ l\ 0.$

Lemma *ord4_nil*: *ord4 nil*.

2.2 Lemas auxiliares de equivalência

2.2.1 Lemas para a prova de **ord1_equiv_ord2**

Para provarmos a equivalência entre as definições indutivas **ord1** e **ord2** (**ord1** $\leftrightarrow$ **ord2**), trataremos uma implicância de cada vez. Primeiro **ord1** $\rightarrow$ **ord2** e, em seguida, **ord2** $\rightarrow$ **ord2**.

Lema **ord1_to_ord2** prova **ord1** $\rightarrow$ **ord2**, ou seja, que toda lista **ord1** também é **ord2**. A base utilizada é baseada na divisão de **ord1** em três regras de formação, executada pela tática **induction** *H*, que divide a prova em três casos:

1. Caso **nil**: utilizada a tática **apply** **ord2_nil**, que prova que uma lista vazia pelas regras de **ord1** também é vazia pelas regras de **ord2**.

2. Caso de um elemento (*x* :: **nil**): prova feita com **ord2_all**, que exige que

- *x* seja menor ou igual a todos os elementos do resto da lista ($x \leq^* \text{nil}$). Como a lista **nil** não tem elementos, isso é uma verdade vazia. A tática **inversion** *Hy* verifica que não faz sentido ter elementos em **nil** e finaliza o problema.
- o resto da lista esteja ordenado (**ord2 nil**), o que é resolvido com **apply** **ord2_nil**.

3. Caso de dois ou mais elementos (*x* :: *y* :: *l*): Sabe-se por hipótese que $x \leq y$ e que o resto da lista (*y* :: *l*) está ordenado. Logo, provar **ord2** (*x* :: *y* :: *l*). Isso se faz ao provar que *x* é menor ou igual a todos os elementos de *y* :: *l* ($x \leq^* y :: l$) quando se aplica **ord2_all**. Isso é possível com auxílio de um elemento qualquer (*z*) dessa lista. Ele pode ser duas coisas e resolvido com **destruct** *H**z*

- Ele é o próprio *y*: Se *z* é *y*, a hipótese $x \leq y$ já resolve o problema (**subst. assumption.**).
- Ele está dentro de *l*: Aqui depende de hipótese de indução (*IHord1*), que garante que *y* :: *l* já é **ord2**. Usando **inversion** sobre essa hipótese, *y* é menor que todos os elementos de *l* ($y \leq z$). Como $x \leq y$ e $y \leq z$, a tática matemática *lia* usa transitividade para concluir que $x \leq z$.

Lemma *ord1_to_ord2* : $\forall l, \text{ord1 } l \rightarrow \text{ord2 } l.$

Lema `ord2_to_ord1` prova o contrário `ord2` $\rightarrow$ `ord1`. Como a `ord2` só possui duas regras de formação, a indução gera apenas dois casos:

1. Caso `nil`: Resolvido com `apply ord1_nil`.
2. Caso de um ou mais elementos (`x :: l`): Sabe-se $x \leq^* l$ e que, por hipótese, a lista `l` atende aos critérios da definição de `ord1`. O problema é resolver o fato de `ord1` ter regras diferentes para listas com 1 e listas com 2+ elementos. Por isso, a tática `destruct l as [| y l']` analisa dois subcasos da própria lista `l`
 - Se `l` for `nil`: Então a lista original é apenas `x`, resolvido com a regra `apply ord1_one`.
 - Se `l` tem elementos (`y :: l'`): Então a lista original é `x :: y :: l'`. A regra que funciona para isso é `ord1_all`, que exige que
 - $x \leq y$: Como `x` é menor ou igual a todos os elementos da lista `y :: l'`, basta aplicar `apply H` apontando que `y` é o primeiro elemento da lista (`left. reflexivity`).
 - `y :: l'` está ordenado em `ord1`: a hipótese de indução já assume isso (`assumption`).

Lemma `ord2_to_ord1` : $\forall l, \text{ord2 } l \rightarrow \text{ord1 } l$.

2.3 Teoremas principais

O objetivo principal desta proposta é mostrar que as definições `ord1`, `ord2`, `ord3` e `ord4` são logicamente equivalentes:

Theorem `ord1_equiv_ord2`: $\forall l, \text{ord1 } l \leftrightarrow \text{ord2 } l$.

Theorem `ord1_equiv_ord3`: $\forall l, \text{ord1 } l \leftrightarrow \text{ord3 } l$.

Theorem `ord1_equiv_ord4`: $\forall l, \text{ord1 } l \leftrightarrow \text{ord4 } l$.

Theorem `ord2_equiv_ord3`: $\forall l, \text{ord2 } l \leftrightarrow \text{ord3 } l$.

Theorem `ord2_equiv_ord4`: $\forall l, \text{ord2 } l \leftrightarrow \text{ord4 } l$.

Theorem `ord3_equiv_ord4`: $\forall l, \text{ord3 } l \leftrightarrow \text{ord4 } l$.

3 Conclusão

Concluir aqui.